

Останин Андрей

Домашнее задание 6: VRP

Первая идея: запустить алгоритм Краскала для кластеризации, а далее дорешать имеющимся у меня TSP солвером (STSP-GA (2-opt), popSize=36, mutProb=0.19, twoOptIters=70 · n + popSize=12, multProb=0.19, twoOptIters=50 · n на $n > 300$ + 2-opt на $n > 10000$).

Таблица 1: Результаты тестирования VRP. Кластеризация (алго Краскала) + TSP-солвер

Название теста	Count	Баллы
vrp_16_3_1	WA	0/5
vrp_26_8_1	WA	0/5
vrp_51_5_1	WA	0/5
vrp_101_10_1	829	5/5
vrp_200_16_1	WA	0/5
vrp_421_41_1	WA	0/5
ИТОГО		5/30

Проблема в моей стратегии набора: я добавляю вершину в кластер, если суммарный вес их не превысит вместимости машины. Однако в конце остаются вершины, которые никуда не деть. Нужно придумать алгоритм кластеризации более надежный.

Сделал рекурсивный перебор ребер в порядке увеличения длины. На каждом уровне рекурсии допускается рассмотреть только k ребер, время работы рекурсии ограничивается $tRec$ секундами.

Таблица 2: Результаты тестирования VRP. Кластеризация (алго Краскала + рекурсивный перебор, $k = 5$, $tRec = 10$) + TSP-солвер

Название теста	Count	Баллы
vrp_16_3_1	315	3/5
vrp_26_8_1	743	3/5
vrp_51_5_1	671	3/5
vrp_101_10_1	829	5/5
vrp_200_16_1	WA	0/5
vrp_421_41_1	2051	3/5
ИТОГО		17/30

Таблица 3: Результаты тестирования VRP. Кластеризация (алго Краскала + рекурсивный перебор, $k = 7$, $tRec = 40$) + TSP-солвер

Название теста	Count	Баллы
vrp_16_3_1	315	3/5
vrp_26_8_1	743	3/5
vrp_51_5_1	WA	0/5
vrp_101_10_1	829	5/5
vrp_200_16_1	WA	0/5
vrp_421_41_1	2051	3/5
ИТОГО		14/30

Как мы видим, нельзя тупо увеличить число рассматриваемых ребер.

Таблица 4: Результаты тестирования VRP. Кластеризация (алго Краскала + рекурсивный перебор, $k = 7$, $tRec = 60$) + TSP-солвер

Название теста	Count	Баллы
vrp_16_3_1	315	3/5
vrp_26_8_1	743	3/5
vrp_51_5_1	671	3/5
vrp_101_10_1	829	5/5
vrp_200_16_1	WA	0/5
vrp_421_41_1	2051	3/5
ИТОГО		17/30

Таблица 5: Результаты тестирования VRP. Кластеризация (алго Краскала + рекурсивный перебор, $k = 7$, $tRec = 600$) + TSP-солвер

Название теста	Count	Баллы
vrp_16_3_1	315	3/5
vrp_26_8_1	743	3/5
vrp_51_5_1	671	3/5
vrp_101_10_1	829	5/5
vrp_200_16_1	WA	0/5
vrp_421_41_1	2051	3/5
ИТОГО		17/30

Я решил остановиться на такой идее, что мой TSP-солвер достаточно хорош, и следует только скормить ему хорошие разбиения точек по машинам. На первый взгляд, весьма резонно объединять точки, которые находятся достаточно близко, и отдавать их одной и той же машине, то есть кластеризовывать. Поэтому теперь нужно решить следующую задачу: набор точек раскидать на заданное число кластеров так, чтобы сумма попарных расстояний/расстояний до центроида кластера была минимальна, и в каждом кластере не превышался порог емкости. Оказалось, что такая проблема действительно существует и называется Capacitated Clustering Problem (но в другой формулировке: максимизируется суммарный профит от ребер в кластерах). Нашел статью Yumin Deng, Jonathan F. Bard (2008) "A reactive GRASP with path relinking for capacitated clustering". Что происходит в статье:

- Стадия 1: выбор изначальных кластеров. Выбираются k (далее k всегда означает число кластеров) вершин с самым большим спросом, они образуют k центров кластеризации. Далее в оригинале на каждом шаге для каждого кластера выбирается максимальное по профиту ребро, инцидентное вершинам из кластера. У меня же понятия максимальной ребра нет, поэтому я буду считать любое доступное ребро (которое не нарушает емкости кластера) максимальным.
- Стадия 2: построение допустимого решения. Оставшиеся свободные вершины последовательно распределяются по кластерам. Выбор вершины в конкретный кластер (при условии, что она в этот кластер помещается) зависит от функции выгоды:

$$gain(u, k) = \sum_{v \in \text{cluster}_k} w_{uv}$$

Для каждой вершины находится ее максимальный $gain(u, k)$. На основе этих значений формируется ограниченный список кандидатов (RCL) вида (вершина, лучший кластер, максимальный выигрыш), куда попадает сколько-то лучших вершин. Длина списка RCL вычисляется как:

$$l_{RCL} = \max(1, \lfloor \alpha \cdot l_{CL} \rfloor)$$

где l_{CL} — общее число доступных кандидатов, а следующая вершина для вставки выбирается из RCL случайно.

α выбирается из набора $\{0.05, 0.1, \dots, 0.5\}$ следующим образом: Каждые несколько десятков итераций вероятности пересчитываются: для каждого значения α_i вычисляется среднее качество полученных решений A_i (сумма попарных весов ребер внутри кластеров), на основе которого рассчитывается ценность соответствующего значения α :

$$q_i = \left(\frac{A_i}{f^*} \right)^\delta$$

где f^* — максимальное качество, полученное с начала работы алгоритма, а $\delta = 10.0$ — коэффициент затухания. Новые вероятности p_i получаются так: $p_i = \frac{q_i}{\sum q_j}$.

- Стадия 3: локальный поиск. Полученное решение пытаемся изменить двумя способами:
 - Попробовать поменять кластер для каждой вершины i на k (если это разрешено и качество увеличивается).
 - Попробовать обменять вершины i и j из разных кластеров (если это разрешено и качество увеличивается).
- Стадия 4: кроссовер. На выходе получаем сколько-то лучших решений, теперь попытаемся их объединить. Для очередной пары решений (пытаемся из одного дойти до другого) находим вершины, у которых назначенные кластеры различные. Если можем бесплатно поменять очередной вершине из этого набора кластер, то меняем и направляем локальный поиск из третьей стадии на промежуточное решение. В конце выбираем лучшее.

Таблица 6: Результаты тестирования VRP. Кластеризация (reactive Grasp, 50 итераций + DSU, если превышен таймаут 1 минута) + TSP-солвер

Название теста	Count	Баллы
vrp_16_3_1	280	5/5
vrp_26_8_1	678	3/5
vrp_51_5_1	616	3/5
vrp_101_10_1	829	5/5
vrp_200_16_1	1754	3/5
vrp_421_41_1	2051	3/5
ИТОГО		22/30

В целом, reactive GRASP справился со своей задачей: я добился, что кластеризация для всех тестов существует, а большего и не ожидалось (так еще и +2 балла выбилось).